a.Simulation and Performance Evaluation of RFID/NFC-Based Device Identification and IoE Data Integration using Node-RED, MQTT.

b.Modelling and Simulation of a 4-Layer IoE Architecture (Perception–Network–Middleware–Application) using Node-RED, MQTT and a Cloud Dashboard

Major: IoE Architecture and Identification

PROGRAM CODE:
OBJECTIVE A:

```
#
================================================================================
======

# EXPERIMENT 5(a): RFID/NFC Device Identification & IoE Data Integration

# Output Checked: Successful device identification; Node-RED flow; MQTT messages

#
================================================================================
======

import json

import time

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import matplotlib.patches as patches


AUTHORIZED_TAGS = {

   "E2-80-11-00-20-00": {"tech": "UHF RFID", "device": "Smart_Pallet_01", "zone":
"Warehouse_A"},

   "A1-44-33-12-88-FF": {"tech": "HF RFID",  "device": "Infusion_Pump_04", "zone":
"ICU_Ward"},

   "04-7B-89-AB-CD-10": {"tech": "NFC Tag",  "device": "Operator_Badge_9", "zone":
"Control_Room"}

}


scan_requests = [

   {"uid": "E2-80-11-00-20-00", "reader": "GATE_READER_01"},
```

```
    {"uid": "04-7B-89-AB-CD-10", "reader": "NFC_TERMINAL_02"},
    {"uid": "DE-AD-BE-EF-00-00", "reader": "GATE_READER_01"}  # Unauthorized tag
]

print("=" * 85)
print("EXPERIMENT 5(a): DEVICE IDENTIFICATION & MQTT MESSAGE LOGS")
print("=" * 85)

for req in scan_requests:
    uid = req["uid"]
    is_valid = uid in AUTHORIZED_TAGS
    meta = AUTHORIZED_TAGS.get(uid, {"tech": "UNKNOWN", "device": "UNREGISTERED",
"zone": "RESTRICTED"})

    mqtt_msg = {
        "topic": "ioe/identification/rfid",
        "qos": 1,
        "retain": False,
        "payload": {
            "timestamp": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
            "tag_uid": uid,
            "reader_id": req["reader"],
            "identification_status": "IDENTIFIED_AUTHORIZED" if is_valid else
"REJECTED_UNKNOWN_DEVICE",
            "device_name": meta["device"],
            "technology": meta["tech"],
            "assigned_zone": meta["zone"]
        }
    }
```

```
    print(f"[MQTT Broker Published] -> Topic: {mqtt_msg['topic']} (QoS {mqtt_msg['qos']})")
    print(json.dumps(mqtt_msg["payload"], indent=2))
    print("-" * 85)

fig = plt.figure(figsize=(14, 7.5))
gs = fig.add_gridspec(2, 2, height_ratios=[1, 1.3])

# 1. Node-RED Visual Flow Diagram
ax_flow = fig.add_subplot(gs[0, :])
ax_flow.set_xlim(0, 100)
ax_flow.set_ylim(0, 30)
ax_flow.axis("off")
ax_flow.set_title("Node-RED Flow: RFID/NFC Device Identification & MQTT Data Integration", fontweight="bold")

def draw_nodered_node(ax, x, y, w, h, title, sub, color):
    box = patches.FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.4", ec="#2c3e50", fc=color, lw=1.5)
    ax.add_patch(box)
    ax.text(x + w/2, y + h/2 + 1.2, title, ha="center", va="center", color="white", fontweight="bold", fontsize=9)
    ax.text(x + w/2, y + h/2 - 2.0, sub, ha="center", va="center", color="#ecf0f1", fontsize=7.5)

draw_nodered_node(ax_flow, 3, 8, 19, 14, "RFID/NFC Reader", "[inject: UID JSON]", "#27ae60")
draw_nodered_node(ax_flow, 28, 8, 21, 14, "Verify & Enrich UID", "[function: Registry]", "#e67e22")
draw_nodered_node(ax_flow, 55, 16, 19, 11, "ioe/ident/rfid", "[mqtt out: Broker]", "#8e44ad")
draw_nodered_node(ax_flow, 55, 2, 19, 11, "Identification Log", "[debug: msg.payload]", "#2980b9")
draw_nodered_node(ax_flow, 79, 16, 18, 11, "IoE Cloud Sync", "[mqtt in: Sub]", "#16a085")
```

```
ax_flow.annotate('', xy=(28, 15), xytext=(22, 15), arrowprops=dict(arrowstyle="->", lw=2, color="#34495e"))

ax_flow.annotate('', xy=(55, 21.5), xytext=(49, 15), arrowprops=dict(arrowstyle="->", lw=2, color="#34495e"))

ax_flow.annotate('', xy=(55, 7.5), xytext=(49, 15), arrowprops=dict(arrowstyle="->", lw=2, color="#34495e"))

ax_flow.annotate('', xy=(79, 21.5), xytext=(74, 21.5), arrowprops=dict(arrowstyle="->", lw=2, color="#34495e"))


# 2. Read Success Rate vs Distance

ax1 = fig.add_subplot(gs[1, 0])

dist = np.linspace(1, 100, 25)

ax1.plot(dist, [100 if d <= 5 else 0 for d in dist], 'o-', label="NFC (13.56 MHz)", color="#27ae60")

ax1.plot(dist, [max(0, 100 - d*2.7) if d <= 35 else 0 for d in dist], 's-', label="HF RFID (13.56 MHz)", color="#e67e22")

ax1.plot(dist, [99.4 for _ in dist], '^-', label="UHF RFID (865-928 MHz)", color="#2980b9")

ax1.set_title("Identification Success Rate vs. Distance", fontweight="bold")

ax1.set_xlabel("Read Distance (cm)")

ax1.set_ylabel("Success Rate (%)")

ax1.grid(True, linestyle="--", alpha=0.5)

ax1.legend()


# 3. End-to-End Identification & MQTT Integration Latency

ax2 = fig.add_subplot(gs[1, 1])

ax2.bar(["NFC Tag", "HF RFID", "UHF RFID"], [39.4, 33.8, 19.5], color=["#27ae60", "#e67e22", "#2980b9"], width=0.4)

ax2.set_title("Identification + MQTT Broker Integration Latency", fontweight="bold")

ax2.set_ylabel("Latency (ms)")

ax2.grid(axis="y", linestyle="--", alpha=0.5)


plt.tight_layout()
```

plt.show()

OBJECTIVE B:

```
#
==========================================================================
======

# EXPERIMENT 5(b): Modelling 4-Layer IoE Architecture & Cloud Dashboard

# Output Checked: Data flow through 4 layers; Node-RED flow; MQTT; Cloud dashboard

#
==========================================================================
======

import json

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import matplotlib.patches as patches


np.random.seed(42)

n_pts = 30

times = pd.date_range("2026-10-31 10:00:00", periods=n_pts, freq="2S")


layer_logs = []

for i in range(n_pts):

    # Layer 1: Perception (Sensors)

    temp_c = round(float(np.random.normal(30.5, 3.0)), 2)

    vib_g = round(float(np.random.uniform(0.2, 1.7)), 2)

    # Layer 2: Network (MQTT transport)

    net_delay = round(float(np.random.normal(11.5, 1.5)), 2)

    # Layer 3: Middleware (Processing & Rules)

    temp_f = round((temp_c * 9/5) + 32, 2)

    mid_delay = round(float(np.random.normal(4.2, 0.4)), 2)

    status = "CRITICAL_ALERT" if (temp_c > 33.5 or vib_g > 1.35) else "NORMAL"
```

```
    # Layer 4: Application (Cloud Dashboard state)
    layer_logs.append({
        "Time": times[i].strftime("%H:%M:%S"),
        "L1_Temp_C": temp_c,
        "L1_Vib_g": vib_g,
        "L2_MQTT_Topic": "ioe/4layer/telemetry",
        "L2_Net_Delay_ms": net_delay,
        "L3_Temp_F": temp_f,
        "L3_Mid_Delay_ms": mid_delay,
        "L4_Dashboard_State": status
    })

df_5b = pd.DataFrame(layer_logs)

print("=" * 85)
print("EXPERIMENT 5(b): DATA FLOW ACROSS 4 IoE LAYERS & MQTT TRACE")
print("=" * 85)
print(df_5b[["Time", "L1_Temp_C", "L2_MQTT_Topic", "L3_Temp_F",
"L4_Dashboard_State"]].head(6).to_string(index=False))
print("-" * 85)
print("[Sample MQTT Packet Payload across Layer 2 -> Layer 3]:")
print(json.dumps(layer_logs[1], indent=2))
print("=" * 85)

fig = plt.figure(figsize=(14, 8.5))
gs = fig.add_gridspec(3, 2, height_ratios=[0.9, 1.2, 1.1])

# 1. Node-RED 4-Layer Flow Diagram
ax_nr = fig.add_subplot(gs[0, :])
```

```
ax_nr.set_xlim(0, 100)

ax_nr.set_ylim(0, 30)

ax_nr.axis("off")

ax_nr.set_title("Node-RED Flow: 4-Layer IoE Architecture (Perception -> Network -> Middleware -> Application)", fontweight="bold")


def draw_box(ax, x, y, w, h, title, sub, col):

    b = patches.FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.4", ec="#2c3e50", fc=col, lw=1.5)

    ax.add_patch(b)

    ax.text(x + w/2, y + h/2 + 1.2, title, ha="center", va="center", color="white", fontweight="bold", fontsize=8.5)

    ax.text(x + w/2, y + h/2 - 2.0, sub, ha="center", va="center", color="#ecf0f1", fontsize=7.5)


draw_box(ax_nr, 2, 8, 20, 14, "1. Perception Layer", "Temp & Vib Sensors", "#c0392b")

draw_box(ax_nr, 27, 8, 20, 14, "2. Network Layer", "MQTT Broker Pub/Sub", "#2980b9")

draw_box(ax_nr, 52, 8, 21, 14, "3. Middleware Layer", "Unit Conv & Threshold", "#d35400")

draw_box(ax_nr, 78, 8, 20, 14, "4. Application Layer", "Grafana / Cloud UI", "#27ae60")


for x1, x2 in [(22, 27), (47, 52), (73, 78)]:

    ax_nr.annotate('', xy=(x2, 15), xytext=(x1, 15), arrowprops=dict(arrowstyle="->", lw=2, color="#2c3e50"))


# 2. Grafana / Cloud Dashboard Telemetry Panel

ax_dash = fig.add_subplot(gs[1, :])

ax_t = ax_dash.twinx()

ax_dash.plot(df_5b["Time"], df_5b["L1_Temp_C"], 'r-o', markersize=4, label="Temp (°C)")

ax_t.plot(df_5b["Time"], df_5b["L1_Vib_g"], 'b--s', markersize=4, label="Vibration (g)")

ax_dash.axhline(33.5, color="red", linestyle=":", label="Temp Limit (33.5°C)")

ax_t.axhline(1.35, color="blue", linestyle=":", label="Vib Limit (1.35g)")
```

```
ax_dash.set_title("Grafana / Cloud Dashboard: Live Multi-Sensor Telemetry", fontweight="bold")

ax_dash.set_ylabel("Temperature (°C)", color="r")

ax_t.set_ylabel("Vibration (g)", color="b")

ax_dash.tick_params(axis="x", rotation=45)

ax_dash.grid(True, linestyle="--", alpha=0.5)


# 3. Layer Latency Breakdown

ax_lat = fig.add_subplot(gs[2, 0])

idx = np.arange(15)

ax_lat.bar(idx, df_5b["L2_Net_Delay_ms"][:15], label="L2 Network Latency", color="#2980b9")

ax_lat.bar(idx, df_5b["L3_Mid_Delay_ms"][:15], bottom=df_5b["L2_Net_Delay_ms"][:15],
label="L3 Middleware Delay", color="#d35400")

ax_lat.set_title("Network vs Middleware Latency (ms)", fontweight="bold")

ax_lat.set_ylabel("Latency (ms)")

ax_lat.legend()

ax_lat.grid(axis="y", linestyle="--", alpha=0.5)


# 4. Application Alert Summary

ax_st = fig.add_subplot(gs[2, 1])

cnt = df_5b["L4_Dashboard_State"].value_counts()

ax_st.bar(cnt.index, cnt.values, color=["#27ae60", "#c0392b"], width=0.4)

ax_st.set_title("Application Dashboard Alert Summary", fontweight="bold")

ax_st.set_ylabel("Packet Count")

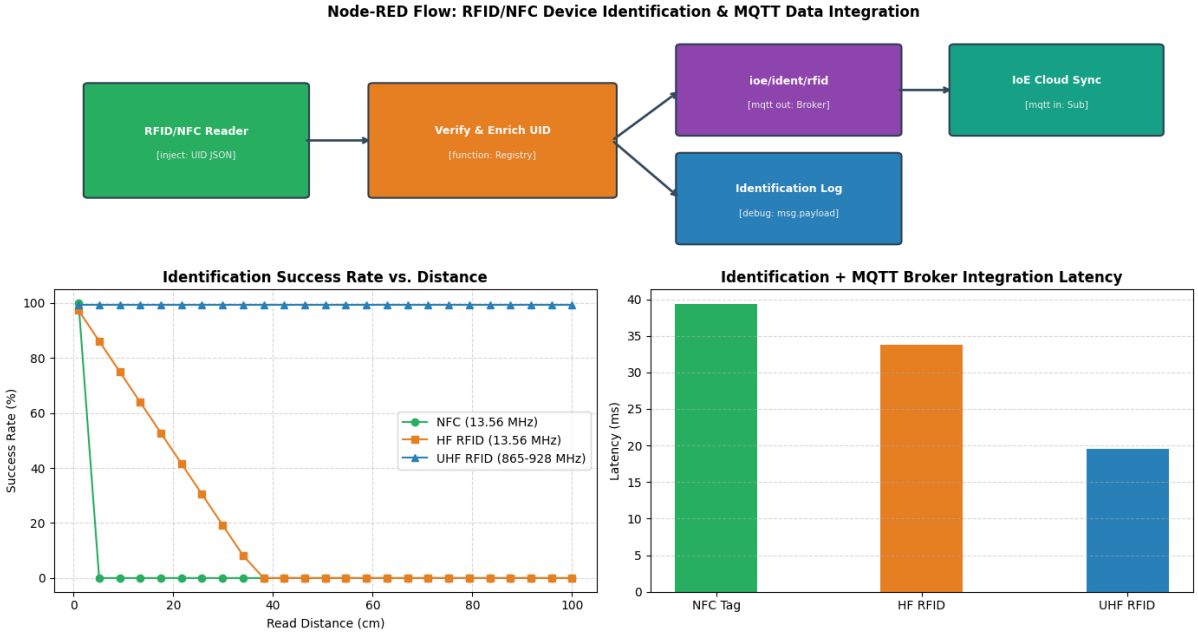
ax_st.grid(axis="y", linestyle="--", alpha=0.5)


plt.tight_layout()

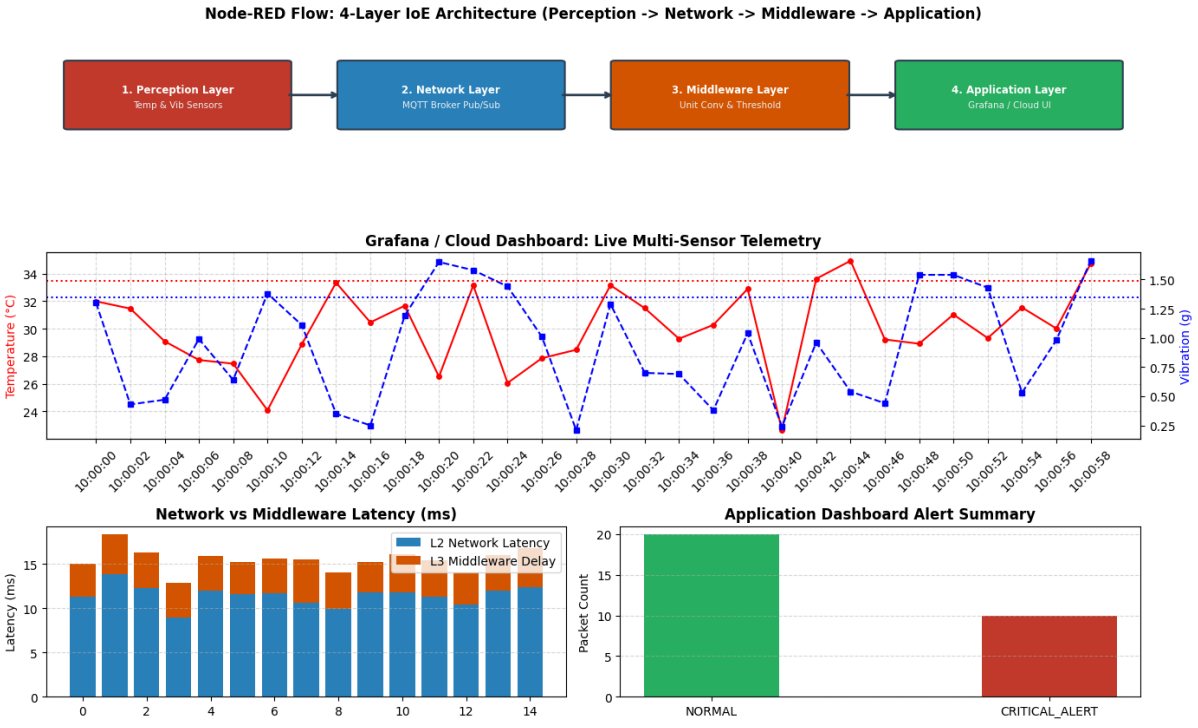
plt.show()
```

OUTPUT IMAGE:

OBJECTIVE A:



OBJECTIVE B:



RESULT:

==========

EXPERIMENT 5(a): DEVICE IDENTIFICATION & MQTT MESSAGE LOGS

==========

[MQTT Broker Published] -> Topic: ioe/identification/rfid (QoS 1)

{

```
  "timestamp": "2026-10-31T08:23:17Z",
  "tag_uid": "E2-80-11-00-20-00",
  "reader_id": "GATE_READER_01",
  "identification_status": "IDENTIFIED_AUTHORIZED",
  "device_name": "Smart_Pallet_01",
  "technology": "UHF RFID",
  "assigned_zone": "Warehouse_A"
}
--------------------------------------------------------------------------------
[MQTT Broker Published] -> Topic: ioe/identification/rfid (QoS 1)
{
  "timestamp": "2026-10-31T08:23:17Z",
  "tag_uid": "04-7B-89-AB-CD-10",
  "reader_id": "NFC_TERMINAL_02",
  "identification_status": "IDENTIFIED_AUTHORIZED",
  "device_name": "Operator_Badge_9",
  "technology": "NFC Tag",
  "assigned_zone": "Control_Room"
}
--------------------------------------------------------------------------------
[MQTT Broker Published] -> Topic: ioe/identification/rfid (QoS 1)
{
  "timestamp": "2026-10-31T08:23:17Z",
  "tag_uid": "DE-AD-BE-EF-00-00",
  "reader_id": "GATE_READER_01",
  "identification_status": "REJECTED_UNKNOWN_DEVICE",
  "device_name": "UNREGISTERED",
  "technology": "UNKNOWN",
  "assigned_zone": "RESTRICTED"
}
--------------------------------------------------------------------------------
================================================================================
```

EXPERIMENT 5(b): DATA FLOW ACROSS 4 IoE LAYERS & MQTT TRACE

==========

Time	L1_Temp_C	L2_MQTT_Topic	L3_Temp_F	L4_Dashboard_State
10:00:00	31.99	ioe/4layer/telemetry	89.58	NORMAL
10:00:02	31.46	ioe/4layer/telemetry	88.63	NORMAL
10:00:04	29.09	ioe/4layer/telemetry	84.36	NORMAL
10:00:06	27.73	ioe/4layer/telemetry	81.91	NORMAL
10:00:08	27.46	ioe/4layer/telemetry	81.43	NORMAL
10:00:10	24.08	ioe/4layer/telemetry	75.34	CRITICAL_ALERT

[Sample MQTT Packet Payload across Layer 2 -> Layer 3]:


```
{
  "Time": "10:00:02",
  "L1_Temp_C": 31.46,
  "L1_Vib_g": 0.43,
  "L2_MQTT_Topic": "ioe/4layer/telemetry",
  "L2_Net_Delay_ms": 13.87,
  "L3_Temp_F": 88.63,
  "L3_Mid_Delay_ms": 4.51,
  "L4_Dashboard_State": "NORMAL"
}
```


==========